

박수민\_0625\_melody 구현

[source code]

#main.c

...

#define F\_CPU 16000000UL

#include <avr/io.h>

#include <util/delay.h>

#include <avr/interrupt.h>

#include "button.h"

#define F\_SCALER 8UL

#define HZ\_TO\_OCR(hz) (F\_CPU / (2UL \* F\_SCALER \* (hz)) - 1)

extern void init\_button(void);

extern int get\_button(int button\_num, int button\_pin);

/\* ---- msec\_count (Timer0 또는 Timer1 인터럽트로 1ms마다 증가) ---- \*/

volatile uint32\_t msec\_count = 0;

extern volatile uint8\_t stop\_music;

extern volatile int stop\_btn;

extern volatile int stop\_pin;

extern void delay\_ms(int ms);

extern void Music\_Player(int \*tone, int \*Beats);

extern void init\_speaker(void);

extern void Beep(int repeat);

extern void Siren(int repeat);

extern void RRR(void);

extern const int Elise\_Tune[];

extern const int Elise\_Beats[];

extern int LG\_Tune[];

extern const int LG\_Beats[];

extern volatile uint8\_t stop\_music;

// PE3 (OC3A) PWM 출력 사용.

// 16bit Timer/Counter

// OCR3A값이 같아지면 Low 출력

void open\_buzzer(void);

```

void power_on_melody(void);
void init_timer0();

/* =====
 * power_on_melody() - 1회 실행분
 * ===== */
void power_on_melody(void)
{
    while(!stop_music)
    {
        OCR3A = HZ_TO_OCR(1000); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(2000); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(3000); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(4000); delay_ms(70);

    }
    OCR3A = 0;
}

/* =====
 * open_buzzer() - 1회 실행분
 * ===== */
void open_buzzer(void)
{
    while(!stop_music)
    {
        OCR3A = HZ_TO_OCR(261); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(329); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(392); delay_ms(70); if(stop_music) break;
        OCR3A = HZ_TO_OCR(554); delay_ms(70);

    }
    OCR3A = 0;
}

/* =====
 * Timer0 초기화 - 1ms 인터럽트용
 * ===== */

ISR(TIMER0_OVF_vect)
{
    TCNT0 = 6;
    msec_count++;
}

```

```

int main(void)
{
    init_timer0();    // msec_count용
    init_button();    // 버튼 핀 입력 설정
    init_speaker();   // Timer3 스피커 설정
    sei();            // 전역 인터럽트 ON

    while(1)
    {
        if(get_button(BUTTON0, BUTTON0PIN))
        {
            stop_btn = BUTTON0; stop_pin = BUTTON0PIN; stop_music = 0;
            power_on_melody();
            OCR3A = 0;
        }
        if(get_button(BUTTON1, BUTTON1PIN))
        {
            stop_btn = BUTTON1; stop_pin = BUTTON1PIN; stop_music = 0;
            open_buzzer();
            OCR3A = 0;
        }
        if(get_button(BUTTON2, BUTTON2PIN))
        {
            stop_btn = BUTTON2; stop_pin = BUTTON2PIN; stop_music = 0;
            Music_Player(LG_Tune, (int*)LG Beats);
            OCR3A = 0;
        }
    }
}

void init_timer0()
{
    // TCNT0 = TimerCount0
    // TCNT0 6 ~ 256 : 250개 펄스를 count 하기 위해 6부터 시작
    TCNT0 = 6;

    TCCR0 &= ~( 1<< CS02 | 1 << CS01 | 1 << CS00); // 0분주 reset 먼저
    TCCR0 |= 1 << CS02 | 0 << CS01 | 0 << CS00;      // 64분주
    TIMSK |= 1 << TOIE0;    // TIMER0 Overflow INT
}

```

```

#Speaker.c
...

/*
 * Speaker.c
 *
 * Created: 2026-06-25 오후 1:50:20
 * Author: kccistc
 */

#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include "button.h"

void delay_ms(int ms);
extern int get_button(int button_num, int button_pin);
void init_speaker(void);
void Music_Player(int *tone, int *Beats);
void Beep(int repeat);
void Siren(int repeat);
int LG_Tune[];
const int LG_Beats[];
volatile int stop_btn = -1; // 지금 재생을 멈출 버튼 (software index)
volatile int stop_pin = -1; // 그 버튼의 핀
#define DO_01 1911
#define DO_01_H 1817
#define RE_01 1703
#define RE_01_H 1607
#define MI_01 1517
#define FA_01 1432
#define FA_01_H 1352
#define SO_01 1276
#define SO_01_H 1199
#define LA_01 1136
#define LA_01_H 1073
#define TI_01 1012
#define DO_02 956
#define DO_02_H 909
#define RE_02 851
#define RE_02_H 804
#define MI_02 758
#define FA_02 716

```

```

#define FA_02_H 676
#define SO_02 638
#define SO_02_H 602
#define LA_02 568
#define LA_02_H 536
#define TI_02 506
#define DO_03 478
#define DO_03_H 450
#define RE_03 425
#define RE_03_H 401
#define MI_03 378

```

```

#define F_CLK 16000000L //클럭
#define F_SCALER 8 //프리스케일러
#define BEAT_1_32 42
#define BEAT_1_16 86
#define BEAT_1_8 170
#define BEAT_1_4 341
#define BEAT_1_2 682
#define BEAT_1 1364

```

// 전역 중단 플래그 추가

```
volatile uint8_t stop_music = 0;
```

```
void Music_Player(int *tone, int *Beats)
```

```
{
```

```
    stop_music = 0; // 시작 시 초기화
```

```
    while(*tone != -1)
```

```
    {
```

```
        if(stop_music) // 중단 요청 시 즉시 탈출
```

```
        {
```

```
            OCR3A = 0;
```

```
            return;
```

```
        }
```

```
        if(*tone == 0)
```

```
        {
```

```
            OCR3A = 0;
```

```
            delay_ms(*Beats);
```

```
        }
```

```
    else
```

```

    {
        OCR3A = *tone;
        delay_ms(*Beats);
        OCR3A = 0;
        _delay_ms(10);
    }
    tone++;
    Beats++;
}
OCR3A = 0;
return;
}

// Timer3 위상교정 PWM
void init_speaker(void)
{
    DDRE |= 0x08;    // PWM CHANNEL OC3A(PE3) 출력 모드로 설정 한다.
    TCCR3A = (1<<COM3A0); // COM3A0 : 비교일치시 PE3 출력 반전 (P328 표14-6 참고)
    TCCR3B = (1<<WGM32) | (1<<CS31); // WGM32 : CTC 4(P327 표14-5) CS31: 8분주(P318)
    // CTC mode : 비교일치가 되면 카운터는 reset되면서 PWM 파형 출력 핀의 출력이 반전 됨.
    // 정상모드와 CTC모드의 차이점은 비교일치 발생시 TCNT1의 레지스터값을 0으로 설정하는지 여부 이다.
    // 정상모드를 사용시 TCNT1이 자동으로 0으로 설정 되지 않아 인터럽트 루틴에서 TCNT1을 0으로 설정 해 주었다.
    // 위상교정 PWM mode4 CTC 분주비 8 16000000/8 ==> 2,000,000HZ(2000KHZ) :
    // up-dounting: 비교일치시 LOW, down-counting시 HIGH출력
    // 1/2000000 ==> 0.0000005sec (0.5us)
    // P599 TOP 값 계산 참고
    // PWM주파수 = OSC(16M) / ( 2(up.down)x N(분주율)x(1+TOP) )
    // TOP = (fOSC(16M) / 2(up.down)x N(분주율)x 원하는주파수 )) -1
    //-----
    // - BOTTOM : 카운터가 0x00/0x0000 일때를 가리킨다.
    // - MAX : 카운터가 0xFF/0xFFFF 일 때를 가리킨다.
    // - TOP?: 카운터가 가질 수 있는 최대값을 가리킨다. 오버플로우 인터럽트의 경우 TOP은 0xFF/0xFFFF
    //          하지만 비교일치 인터럽트의 경우 사용자가 설정한 값이 된다.

    TCCR3C = 0; // P328 그림 14-11 참고
    OCR3A = 0; // 비교 일치값을 OCR3A에 넣는다.
}

```

```

        return;
    }

void Beep(int repeat)
{
    int i;

    for(i=0; i < repeat; i++)
    {
        OCR3A = 500; // 0.00025sec (250us) : 0.0000005 * 500
        _delay_ms(200);
        OCR3A = 0;
        _delay_ms(200);
    }
}

void Siren(int repeat)
{
    int i, j;

    OCR3A = 900;

    for(i=0; i < repeat; i++)
    {
        for(j=0; j < 100; j++)
        {
            OCR3A += 10;
            _delay_ms(20);
        }
        for(j=0; j < 100; j++)
        {
            OCR3A -= 10;
            _delay_ms(20);
        }
    }
}

void RRR(void)
{
    int i;

    for(i=0; i<20; i++)
    {

```

```

        OCR3A = 1136;
        _delay_ms(100);
        OCR3A = 0;
        _delay_ms(20);
    }
}

void delay_ms(int ms)
{
    while(ms-- != 0)
    {
        _delay_ms(1);
        if(stop_btn >= 0 && get_button(stop_btn, stop_pin))
            stop_music = 1;    // 재생 중 그 버튼 또 누르면 중단
    }
}

int LG_Tune[] = {
    RE_02,
    SO_02, FA_02_H, MI_02,    // 솔 파# 미b
    RE_02, TI_01,            // 레 시
    DO_02, RE_02, MI_02,    // 도레미
    LA_01, TI_01, DO_02,    // 라시도
    TI_01, RE_02,            // 시레

    RE_02,                    // 레
    SO_02, FA_02_H, MI_02,    // 솔파#미
    RE_02,                    // 레
    SO_02,                    // 솔
    SO_02, LA_02, SO_02,    // 솔라솔
    FA_02_H, MI_02, FA_02_H, // 파# 미 파#
    SO_02,                    // 솔

    DO_02,
    FA_02, MI_02, RE_02,
    DO_02, LA_01,
    LA_01_H, DO_02, RE_02,
    SO_01, LA_01, LA_01_H,
    LA_01,
    DO_02,

    /* 도 파미레 도 파 파솔파 미레미 파 */

```



```

DO_02,
FA_02, MI_02, RE_02,
DO_02, FA_02,
FA_02, SO_02, FA_02,
MI_02, RE_02, MI_02,
FA_02,

-1
};

const int LG Beats[] = {
    BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4, BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4, BEAT_1_4,

    BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4, BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_2,

    BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4, BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4,
    BEAT_1_4,

    /* 2번째 줄 */
    BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_4, BEAT_1_4,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_8, BEAT_1_8, BEAT_1_8,
    BEAT_1_2,
};

```